

FICHAMENTO ACADÊMICO TÉCNICO

REFERÊNCIA COMPLETA: FALEIROS, Fabiano Reis; FERREIRA, Hiran Nonato Macedo. **Um estudo de arquiteturas, design patterns e boas práticas de codificação: vantagens e desvantagens.** In: 14ª Jornada Científica e Tecnológica e 11º Simpósio de Pós-Graduação do IFSULDEMINAS, 2022, Passos. Anais... Passos: IFSULDEMINAS, 2022. Disponível em: <https://josif.ifsuldeminas.edu.br/ojs/index.php/anais/article/view/614/235>. Acesso em: 20 mar. 2026.

IDEIAS PRINCIPAIS:

- **Escalabilidade e Manutenibilidade:** O artigo foca na necessidade de desenvolver softwares que suportem o crescimento exponencial de usuários (escalabilidade) e que sejam fáceis de manter a longo prazo (manutenibilidade), reduzindo o débito técnico e o tempo gasto em novas funcionalidades.
 - **Arquitetura e Design Patterns:** A utilização de padrões como Domain Driven Design (DDD), Dependency Injection (DI) e Repository/Service Pattern é apresentada como essencial para organizar o código em camadas, facilitando a testabilidade e a evolução do sistema.
 - **Código Limpo (Clean Code):** A prática de Clean Code é tratada não apenas como uma regra técnica, mas como uma filosofia de desenvolvimento que visa a expressividade e a simplicidade, garantindo que o código seja legível tanto para máquinas quanto para humanos.
 - **Integração e Entrega Contínua (CI/CD):** O estudo propõe a aplicação prática desses conceitos através de uma API REST (Node.js) e um cliente React, utilizando pipelines de CI/CD para garantir a qualidade e a velocidade na entrega de software.
-

RESUMO:

O presente trabalho apresenta um relato de pesquisa focado na construção de softwares escaláveis e de alta qualidade através da combinação de arquiteturas modernas, padrões de projeto e boas práticas de codificação. O ponto de partida é o cenário atual de crescimento massivo de usuários na internet, o que exige aplicações robustas que atendam a demandas cada vez maiores sem perda de desempenho.

Os autores exploram a fundamentação teórica de conceitos como escalabilidade (capacidade de crescer conforme a carga), manutenibilidade (facilidade de alteração) e modularidade. Destacam o papel do *Clean Code* como um facilitador para o trabalho em equipe, permitindo que novos desenvolvedores compreendam o sistema rapidamente. A metodologia envolve o desenvolvimento de uma plataforma voltada à saúde mental, servindo como prova de conceito para a aplicação de padrões como DDD e Injeção de Dependência. O estudo conclui que a adoção conjunta dessas técnicas não apenas melhora a experiência do usuário final, mas também otimiza os processos internos de desenvolvimento, garantindo segurança em alterações futuras através de testes unitários e automação.

CITAÇÕES DO ARTIGO:

- “Não há uma definição tão clara a respeito do código limpo, Almeida e Miranda (2010) citam que a escrita de códigos pode ser comparada com a arte, pois não é possível definir parâmetros lógicos e mensuráveis que definam a diferença de qualidade entre códigos-fonte.” (FALEIROS; FERREIRA, 2022, p. 2)
 - “Para Almeida e Miranda (2010) e Martin (2011) um código limpo se trata de um estilo de programação próximo a três valores: Expressividade, Simplicidade e Flexibilidade.” (FALEIROS; FERREIRA, 2022, p. 2)
-

ANÁLISE TÉCNICA (PERFORMANCE E QUALIDADE):

- O artigo demonstra uma preocupação central com a **performance estrutural** do software. Ao defender o uso de *Design Patterns*, os autores visam evitar o “código espaguete”, que degrada o desempenho da equipe e aumenta a latência na entrega de novas *features*.

- A menção a **Testes Unitários** como “fundação para a metodologia” é um ponto crítico para a performance de longo prazo, pois evita regressões que poderiam comprometer a estabilidade do sistema sob carga.
- A escolha do *stack* tecnológico (Node.js e React) aliada a práticas de CI/CD reflete uma abordagem moderna voltada para a **eficiência operacional**, onde a automação reduz o erro humano e acelera o ciclo de feedback.